University of Asia Pacific

Department of Computer Science and Engineering

CSE 402: Operating System Lab

LAB REPORT

Submitted by: Shad Bin Moshiur

Student ID: 23101143

Section: C-2

Lab task: 4

Submitted to:

Atia Rahman Orthi
Lecturer,
Department of Computer Science and Engineering

Date of Submission: 17/08/2026

LAB REPORT 4

Round Robin CPU Scheduling

(with Comparative Analysis against FCFS and SJF)

Operating System Lab | Student ID: 23101143

---

1. Objective

To implement the Round Robin (RR) CPU scheduling algorithm in Python and to compare its performance against First Come First Serve (FCFS) and Shortest Job First (SJF, non-preemptive) using Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) as evaluation metrics.

2. Theory

2.1 First Come First Serve (FCFS)

FCFS is a non-preemptive scheduling algorithm in which processes are executed strictly in the order of their arrival. It is simple to implement but can suffer from the convoy effect, where a long process delays all shorter processes waiting behind it.

2.2 Shortest Job First (SJF) — Non-Preemptive

SJF selects the process with the smallest burst time among all processes that have arrived and are ready to execute. Once selected, the process runs to completion without interruption. SJF minimizes average waiting time but can cause starvation of longer processes if shorter processes keep arriving.

2.3 Round Robin (RR)

Round Robin is a preemptive scheduling algorithm designed for time-sharing systems. Each process in the ready queue is given a fixed time slice called a quantum. If a process does not finish within its quantum, it is preempted and placed at the back of the ready queue, and the CPU moves to the next process. RR guarantees fairness and eliminates starvation, but its average TAT and WT depend heavily on the chosen quantum size.

3. Process Set Used

The following process set (Arrival Time and Burst Time in ms) was used for all three algorithms, with a time quantum of 5 ms for Round Robin:

PID	Arrival Time (AT)	Burst Time (BT)
P1	0	7
P2	1	4
P3	2	15
P4	3	11
P5	4	20

PID	Arrival Time (AT)	Burst Time (BT)
P6	4	9

Table 1: Input process set (AT = Arrival Time, BT = Burst Time)

4. Implementation Summary

All three algorithms were implemented in Python:

- fcfs() — sorts processes by arrival time and schedules them sequentially.
- sjf_non_preemptive() — at each decision point, picks the arrived process with the smallest remaining burst time.
- round_robin() — maintains a ready queue (Python deque); processes are pushed in as they arrive, executed for at most one quantum, and re-queued if unfinished.

For every process, Completion Time (CT), Turnaround Time (TAT = CT − AT), and Waiting Time (WT = TAT − BT) were calculated, and the average TAT and WT were computed for each algorithm.

5. Results

5.1 FCFS Result

PID	AT	BT	CT	TAT	WT
P1	0	7	7	7	0
P2	1	4	11	10	6
P3	2	15	26	24	9
P4	3	11	37	34	23
P5	4	20	57	53	33
P6	4	9	66	62	53
Average	-	-	-	**31.67**	**20.67**

Table 2: FCFS scheduling result

5.2 SJF (Non-Preemptive) Result

PID	AT	BT	CT	TAT	WT
P1	0	7	7	7	0
P2	1	4	11	10	6
P3	2	15	46	44	29
P4	3	11	31	28	17
P5	4	20	66	62	42
P6	4	9	20	16	7
Average	-	-	-	**27.83**	**16.83**

Table 3: SJF (non-preemptive) scheduling result

5.3 Round Robin Result (Quantum = 5)

PID	AT	BT	CT	TAT	WT
P1	0	7	31	31	24
P2	1	4	9	8	4
P3	2	15	55	53	38
P4	3	11	56	53	42
P5	4	20	66	62	42
P6	4	9	50	46	37
Average	-	-	-	**42.17**	**31.17**

Table 4: Round Robin scheduling result

5.4 Comparison Summary

Metric	FCFS	SJF (Non-Preemptive)	Round Robin (Q=5)
Average TAT	31.67	27.83	42.17
Average WT	20.67	16.83	31.17

Table 5: Average TAT and WT comparison across FCFS, SJF, and RR

6. Discussion

Among the three algorithms, SJF (non-preemptive) produced the lowest average TAT (27.83 ms) and lowest average WT (16.83 ms) for this process set, since it always prioritizes the shortest available job, minimizing the time other short processes spend waiting. FCFS performed moderately (average TAT 31.67 ms, average WT 20.67 ms); because processes were scheduled strictly in arrival order, the long burst time of P3 (15 ms) delayed P4, P5, and P6 behind it, an example of the convoy effect.

Round Robin produced the highest average TAT (42.17 ms) and average WT (31.17 ms) among the three. With a quantum of 5 ms, long processes such as P3 (15 ms), P4 (11 ms), and P5 (20 ms) had to be executed in multiple rounds, repeatedly cycling back through the ready queue. This repeated preemption increases the completion time of longer jobs even though it improves fairness and response time for shorter ones. P2, with the smallest burst time (4 ms), finished fastest under RR (CT = 9), showing that RR is favorable for short, interactive processes even though the overall averages are pulled up by the longer processes in this set.

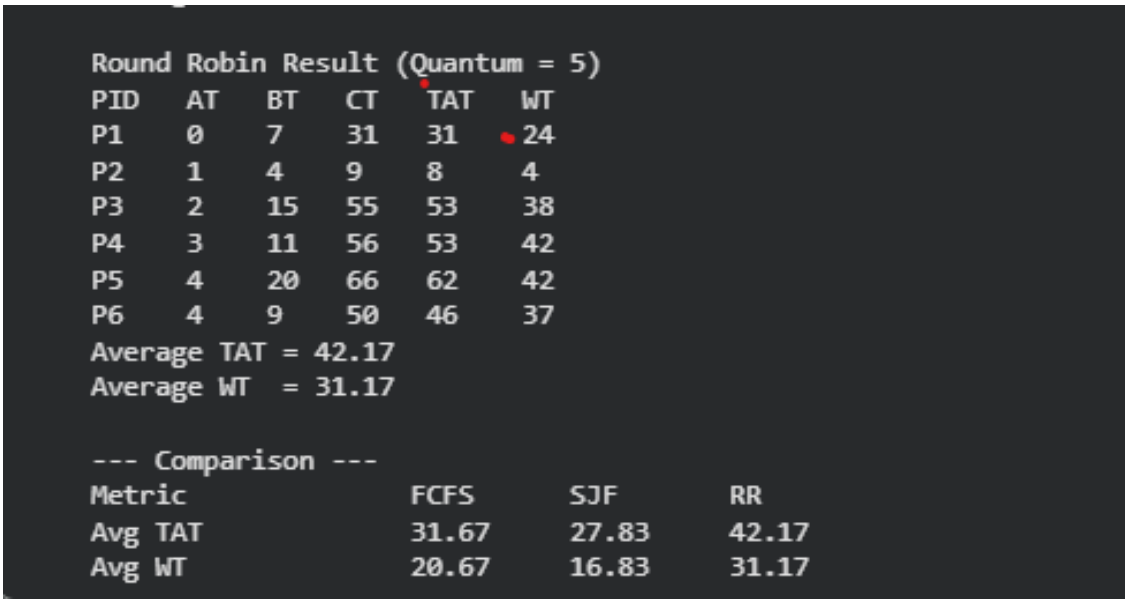
It should be noted that SJF requires prior knowledge of burst times, which is rarely available in real operating systems, and it can starve longer processes if shorter ones keep arriving. RR does not require this knowledge and guarantees that every process gets CPU time within a bounded interval, making it more practical for general-purpose and time-sharing systems, even though it does not minimize average TAT/WT for this particular data set.

7. Conclusion

FCFS, SJF, and Round Robin were implemented and compared using the same process set. SJF achieved the best average TAT and WT but is impractical without prior burst-time knowledge and risks starvation. FCFS is simple but suffers from the convoy effect when a long process arrives early. Round Robin, despite having the highest average TAT and WT for this data set with a quantum of 5 ms, ensures fairness and bounded waiting time for all processes, making it the most suitable choice for real-time, interactive, and time-sharing operating systems. The choice of scheduling algorithm therefore depends on the system's priorities — throughput and average waiting time (favoring SJF), simplicity (favoring FCFS), or fairness and responsiveness (favoring Round Robin).

8. Program Output

The screenshot below shows the actual console output of the Round Robin result and the FCFS vs SJF vs RR comparison, produced by running the script in Section 9.



```
Round Robin Result (Quantum = 5)
PID   AT   BT   CT   TAT   WT
P1    0    7    31   31    24
P2    1    4    9    8     4
P3    2    15   55   53    38
P4    3    11   56   53    42
P5    4    20   66   62    42
P6    4    9    50   46    37
Average TAT = 42.17
Average WT  = 31.17

--- Comparison ---
Metric              FCFS      SJF       RR
Avg TAT             31.67     27.83     42.17
Avg WT              20.67     16.83     31.17
```

Figure 1: Console output — Round Robin result and algorithm comparison

GithubLink:
https://github.com/Moshiur1143/Operating-System-Repository-23101143-/tree/main/Lab-04-Round-Robin-Cpu-Scheduling/Report

9. Appendix: Python Source Code

The complete Python implementation used for this lab (FCFS, SJF non-preemptive, and Round Robin) is listed below:

```
# -*- coding: utf-8 -*-
"""ROUND ROBIN CPU Scheduling.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/15PnLtFfRl5_cRrXEkoe4BPl-4Ih1kGIW
"""

processes = [
    ["P1", 0, 7],
    ["P2", 1, 4],
    ["P3", 2, 15],
    ["P4", 3, 11],
    ["P5", 4, 20],
    ["P6", 4, 9]
]



def fcfs(processes):
    procs = sorted(processes, key=lambda x: x[1])

    current_time = 0
    result = []

    for pid, at, bt in procs:

        if current_time < at:
            current_time = at

        ct = current_time + bt
        tat = ct - at
        wt = tat - bt

        result.append([pid, at, bt, ct, tat, wt])

        current_time = ct

    return result



def sjf_non_preemptive(processes):

    n = len(processes)
    is_done = [False] * n

    current_time = 0
    completed = 0
    result = []

    while completed != n:

        idx = -1
        min_bt = float('inf')

        for i in range(n):

            pid, at, bt = processes[i]

            if at <= current_time and not is_done[i]:

                if bt < min_bt:
                    min_bt = bt
                    idx = i
```

```
        if idx == -1:
            current_time += 1
            continue

        pid, at, bt = processes[idx]

        current_time += bt

        ct = current_time
        tat = ct - at
        wt = tat - bt

        result.append([pid, at, bt, ct, tat, wt])

        is_done[idx] = True
        completed += 1

    return result



def round_robin(processes, quantum):

    n = len(processes)

    procs = sorted(processes, key=lambda x: x[1])

    remaining_bt = [p[2] for p in procs]

    completion_time = [0] * n

    ready_queue = deque()

    current_time = 0
    i = 0

    while i < n or ready_queue:


        while i < n and procs[i][1] <= current_time:
            ready_queue.append(i)
            i += 1


        if not ready_queue:
            current_time = procs[i][1]
            continue


        idx = ready_queue.popleft()

        execution_time = min(quantum, remaining_bt[idx])

        current_time += execution_time

        remaining_bt[idx] -= execution_time

        while i < n and procs[i][1] <= current_time:
            ready_queue.append(i)
            i += 1


        if remaining_bt[idx] > 0:
            ready_queue.append(idx)

        else:
            completion_time[idx] = current_time

    result = []

    for i in range(n):

        pid, at, bt = procs[i]

        ct = completion_time[i]
```

```
        tat = ct - at
        wt = tat - bt

        result.append([pid, at, bt, ct, tat, wt])

    return result


def print_table(title, result):

    print("\n" + title)

    print(f"{'PID':<6}{'AT':<5}{'BT':<5}{'CT':<5}{'TAT':<6}{'WT':<5}")

    for r in sorted(result, key=lambda x: x[0]):

        pid, at, bt, ct, tat, wt = r

        print(f"{pid:<6}{at:<5}{bt:<5}{ct:<5}{tat:<6}{wt:<5}")

    avg_tat = sum(r[4] for r in result) / len(result)
    avg_wt = sum(r[5] for r in result) / len(result)

    print(f"Average TAT = {avg_tat:.2f}")
    print(f"Average WT  = {avg_wt:.2f}")

    return avg_tat, avg_wt


fcfs_result = fcfs(processes)

sjf_result = sjf_non_preemptive(processes)

quantum = 5
rr_result = round_robin(processes, quantum)



fcfs_tat, fcfs_wt = print_table(
    "FCFS Result",
    fcfs_result
)

sjf_tat, sjf_wt = print_table(
    "SJF Result",
    sjf_result
)

rr_tat, rr_wt = print_table(
    f"Round Robin Result (Quantum = {quantum})",
    rr_result
)



print("\n--- Comparison ---")

print(f"{'Metric':<20}{'FCFS':<10}{'SJF':<10}{'RR':<10}")

print(
    f"{'Avg TAT':<20}"
    f"{fcfs_tat:<10.2f}"
    f"{sjf_tat:<10.2f}"
    f"{rr_tat:<10.2f}"
)

print(
    f"{'Avg WT':<20}"
    f"{fcfs_wt:<10.2f}"
    f"{sjf_wt:<10.2f}"
    f"{rr_wt:<10.2f}"
)
```

Note: this is the script as exported from Google Colab. `deque` is used in round_robin() via `from collections import deque`, which was run in an earlier notebook cell and should be included when running this file standalone.